Makefile 撰寫

簡單記錄基本的 makefile 撰寫方法 以前都靠 GPT 幫忙寫 一直都沒搞懂怎麼去寫這個東西

1. 基本

一個 **Makefile** 的本質是：

> 定義「目標（target）」如何從「來源（dependencies）」產生出來的規則（rule）。

基本格式是：

```
target: dependencies
<TAB> command
```

注意：`command` **前面一定要用 TAB 鍵**（不是空格）。

```
hello: hello.cpp
    g++ -o hello hello.cpp
```

執行 `make hello` 時會：

1. 發現 `hello` 這個目標需要 `hello.cpp`
2. 執行命令 `g++ -o hello hello.cpp`

通常我們會把編譯指令或者 target 等等寫成變數方便修改

```
CXX = g++
CXXFLAGS = -std=c++17 -O2 -Wall
```

```
SRC = hello.cpp
TARGET = hello
```

然後規則就可以寫成：

```
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) -o $@ $^
```

- `$@` → 代表目前的「目標名稱」（這裡是 `hello`）
- `$^` → 代表「所有相依的檔案」（這裡是 `hello.cpp`）

那如果單純只輸入一個 make 呢？

`make` 會執行 **Makefile 中的第一個 target**。

範例：

```
hello: hello.cpp
    g++ -o hello hello.cpp

clean:
    rm -f hello
```

執行：

```
make
```

會執行第一個 target，也就是 `hello`。

執行：

```
make clean
```

則會執行 `clean` 這個 target。

注意

如果你的 Makefile 開頭長這樣：

```
clean:
    rm -f hello

hello: hello.cpp
    g++ -o hello hello.cpp
```

那你輸入 `make` 時會先執行 `clean` ！

這通常不是你想要的行為

所以在實務上，大家習慣把主要目標放第一個，例如：

```
all: hello

hello: hello.cpp
    g++ -o hello hello.cpp

clean:
    rm -f hello
```

然後讓使用者打 `make` 時，預設會執行「主要的」目標（例如 `all`）。這邊每個 target 的 dependence 可以是其他的 target 這樣一直遞迴下去是可行的，所以實務上通常會用一個 `all` 的 target 來當作全部要一起執行用到的 target

---

小結論

行為	說明
`make`	執行第一個 target
`make <target>`	執行指定的 target
常見慣例	第一個 target 通常取名為 `all`

這邊再說明一下 dependence 到底是什麼，回顧一下這邊的語法

```
target: dependencies
    commands
```

意思是：

> 要生成 target，需要先確保 dependencies 都是「最新的」；
>
> 如果不是，就執行底下的 `commands`。

白話一點的語意是：

> hello 這個檔案，是根據 hello.cpp 產生出來的。
>
> 當 `hello.cpp` 變動時，`make` 應該重新編譯。

而假設不寫改寫成這樣：

```
hello:
    g++ -o hello hello.cpp
```

這樣也能用，執行 `make` 一樣會編出 `hello`。

但有個重要差別：

- `make` 不知道 `hello` 跟 `hello.cpp` 有關
- 所以當你修改 `hello.cpp` 之後再次執行 `make`，

  它**不會自動重新編譯**，因為 `make` 會以為「`hello` 已經是最新的檔案」。

  `make: 'hello' is up to date.` 會顯示出這個，因為執行檔 hello 已存在，當然你如果手動刪掉再 make 還是可以重新編譯

簡單來說 寫依賴的好處就是當你的依賴檔案一有更動，不管原本的執行檔存不存在都可以 make 去編譯

2. 多檔案

用這次寫 PDA 作業的目錄結構來舉例

```
HW3/
├─ src/
│  ├─ core/
│  │  ├─ design.hpp        // 共用資料模型（Sites, Macros, Rows, Insts, Nets...）
│  │  ├─ design.cpp
│  │  ├─ units.hpp         // 單位轉換工具、snapping helper
│  │  └─ hpwl.hpp          // HPWL 與 bbox 計算
│  ├─ io/
│  │  ├─ lef_reader.hpp    // LEF 解析器介面（不依賴 placer）
│  │  ├─ lef_reader.cpp
│  │  ├─ def_reader.hpp    // DEF 解析器介面
│  │  ├─ def_reader.cpp
│  │  ├─ def_writer.hpp    // 輸出 DEF（只改 COMPONENTS 的座標/方向）
│  │  └─ def_writer.cpp
│  ├─ placer/
│  │  ├─ detailed_placer.hpp // 你的演算法（local reorder / swap / ISM）
│  │  └─ detailed_placer.cpp
│  ├─ main.cpp
│  └─ Makefile
└─ bin/ output/
```

最簡單的編譯方式當然就類似下面這樣

```
$(TARGET): main.cpp io.cpp placer.cpp
   $(CXX) $(CXXFLAGS) -o $@ $^
```

這樣當然是可以運行的．但有個缺點，假設你某次改動只改 io.cpp，但依舊會把 main.cpp, io.cpp, placer.cpp 全部重新編譯一次，因為只要某個 dependence 有改變了就會觸發重新編譯的條件導致全部重新編譯

所以當你有多個 `.cpp` 檔時，不想每次都重編全部，我們可以拆成兩步：

1. 先把每個 `.cpp` 編成 `.o`
2. 再把所有 `.o` 連結成執行檔

先描述物件檔清單：

```
SRC = main.cpp io.cpp placer.cpp app.h
OBJ = $(SRC:.cpp=.o)
```

這行的意思是：

把 `SRC` 裡面的每個 `.cpp` 換成 `.o` → `main.o io.o placer.o` 但 `app.h` 因為不符合後綴所以不會變，所以上面那兩行可以視為

```
SRC = main.cpp io.cpp placer.cpp app.h
OBJ = main.o io.o placer.o app.h
```

接下來就要把 .cpp 去編譯成 .o 檔了，比較簡單的方法可以用下面這種方式

```
%.o: %.cpp
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

這句在描述所有 .cpp 檔都會經由這種方式編譯成 .cpp 檔

現在已經成功地讓其他每個 .cpp 都能自動編譯成連結檔案了，目的就是為了不用每次有某個檔案改動重新編譯時就要重編所有檔案，下面要說明如果是 .hpp 修改了那要怎麼辦

首先在 `CXXFLAGS` 後面加兩個旗標

```
CXXFLAGS += -MMD -MP
```

- `MMD`：為每個 .o 產生一個同名 `.d`（記錄此 .cpp 直接/間接包含了哪些 .hpp）
- `MP`：避免刪檔造成的假相依錯誤

建 `.d` 清單並 include 進來（放在檔案靠後處、規則之上或之下都行）

```
DEPS = $(OBJ:.o=.d)
-include $(DEPS)
```

自動掃描所有 `.cpp`，不再手動維護 `SRC`

把原本人工列的

```
SRC = \
  main.cpp \
  core/design.cpp \
  io/lef_reader.cpp \
  io/def_reader.cpp \
  io/def_writer.cpp \
  placer/detailed_placer.cpp
```

改成

```
SRC = $(wildcard core/*.cpp io/*.cpp placer/*.cpp) main.cpp
```

> 註：main.cpp 放在 src/ 根目錄，所以額外寫上。

最後只要再把 .o 檔案 link 起來編譯成 target ，全部的工作就結束了

```
$(TARGET): $(OBJ)
    $(CXX) $(CXXFLAGS) $(OBJ) -o $@
```

額外補充 `make run` 跟 `.PHONY`

什麼是 `.PHONY`？

`Makefile` 裡面每一個「目標」(target)，

原本都是「**檔案名稱**」的意思。

例如：

```
main.o: main.cpp
```

意思是「如果 `main.o` 這個檔案不存在或比 `main.cpp` 舊，就重編」。

但是有時候，我們會定義一些「不是檔案」的目標，

像是：

```
clean:
    rm -rf *.o
```

這裡的 `clean` 並不是要建立一個名為 `clean` 的檔案，

而只是「我想要執行一個清理動作」。

問題是：

如果你的資料夾裡剛好真的出現一個叫做 `clean` 的檔案，

`make clean` 就會以為這個檔案「已經存在而且是最新的」，

所以完全不執行那個指令！

解法：宣告為 `.PHONY`

意思是「這個 target 不是檔案名稱，而是一個純動作」。

```
.PHONY: clean run
```

這樣 `make` 就知道：

> 「不管目錄裡有沒有叫 clean 或 run 的檔案，都要執行這段規則。」

而在很多大專案裡面都會有 run 這個 target

```
run: $(TARGET)
    @$(TARGET)
```

這是一個「目標規則」：

意思是：

1. 這個規則的名字叫 `run`。
2. 它**依賴** `$(TARGET)`（也就是你的可執行檔，例如 `../bin/hw3_placer`）。
   - 所以在執行 `make run` 前，它會先確保程式有被編好。
3. 接著執行底下那一行指令（縮排必須是 Tab！）：

   ```
   @$(TARGET)
   ```

   `@` 的意思是「執行這個指令，但不要把指令本身印出來」。

實際效果

```
$ make run
g++ ...  # 如果程式還沒編會先編
../bin/hw3_placer
```

如果程式已經是最新的，就只會執行：

```
../bin/hw3_placer
```

等於是「**編譯並執行**」的快捷方式。

附上這次 homework3 最簡單的 makefile

```
CXX = g++
CXXFLAGS = -O3 -MMD -MP
TARGET = ../bin/main
```

```
SRC = $(wildcard core/*.cpp io/*.cpp placer/*.cpp) main.cpp
OBJ = $(SRC:.cpp=.o)
DEPS = $(OBJ:.o=.d)
-include $(DEPS)

.PHONY: all
all: $(TARGET)

$(TARGET): $(OBJ)
    $(CXX) $(CXXFLAGS) $(OBJ) -o $@

%.o: %.cpp
    $(CXX) $(CXXFLAGS) -c $< -o $@

.PHONY: run clean
run: $(TARGET)
    @$(TARGET)

clean:
    rm -f $(OBJ) $(DEPS) $(TARGET)
```